

Busqueda en arboles de juego

Teoría de juegos $\leftarrow$ Teoría de Nash

Juegos deterministas $\leftarrow$ Sin azar

Juegos estocásticos $\leftarrow$ interviene el azar

Teorema Minimax $\leftarrow$ En juegos con intereses completamente opuestos existe una forma racional de actuar

$\uparrow$
juegos de suma cero

Algoritmo Minimax

1. Se construye el árbol completo del juego, uso de generador de movimientos (expansión)
2. Evaluamos el estado del tablero en los nodos hoja utilizando una función de evaluación (heurística).
3. Recorre el árbol desde los nodos hoja hasta el nodo raíz considerando el tipo de jugador:
 - a) Para jugador MAX: seleccionamos el nodo hijo con el valor máximo
 - b) Para jugador MIN: seleccionamos el nodo hijo con el valor mínimo.
4. En el nodo raíz nos quedamos con el nodo hijo de valor máximo y realizamos el movimiento correspondiente.

Características

La función de evaluación (heurística)

$$\text{Heurística}(\text{tablero}) = \text{valor}(\text{máquina}) - \text{valor}(\text{humano})$$

Podar Alfa-Beta $\leftarrow$ Reduce el número de nodos evaluados por Minimax

α = mejor opción hasta el momento para MAX (valor más alto)

β = mejor opción hasta el momento para MIN (valor más bajo).

si en algún momento $\alpha \geq \beta$ se puede podar

Intervalo mejores jugadas encontrados $(-\infty, \infty)$

$\uparrow \quad \uparrow$
MAX MIN

Exploración en espacios de estados

↑ Representación del espacio como grafos

Estado $\leftarrow$ configuración concreta del problema.

Transición $\leftarrow$ regla que permite pasar de un estado a otro.

Sucesores $\leftarrow$ estados alcanzables desde uno dado aplicando transiciones.

Camino $\leftarrow$ secuencia de estados conectados por transiciones.

Expansión de un camino $\leftarrow$ tomar los sucesores del último estado y generar nuevos caminos (evitando ciclos).

Espacio implícito Vs explícito $\leftarrow$ normalmente no construimos todo el grafo.

Diseño de un sistema de búsqueda

1. Representación de estados (estructura de datos).

2. Función $\text{successors}(\text{state}) \rightarrow$ lista de estados alcanzables.

3. Función $\text{solution}(\text{path}/\text{state}) \rightarrow$ indica si es solución.

4. (Optional) $\text{cost}(\text{path})$ y $\text{heuristic}(\text{state})$ si busco óptimos o uso búsqueda informada.

5. Elegir el algoritmo de búsqueda según objetivo (solución rápida, camino óptimo, memoria, etc.).

Algoritmo general

- Mantener una lista de caminos pendientes (CP). Inicialmente contiene el camino con el estado inicial.

- Repetir: extraer primer camino C de CP; si C es solución $\rightarrow$ terminar; sino expandir C (generar E) y actualizar CP según la política del algoritmo.

- Diferencia entre algoritmos = cómo se reordenan/mezclan CP y E.

Búsqueda no informada (búsqueda ciega) (fuerza bruta)

- Búsqueda de una solución: Exploración en amplitud y en profundidad.

Amplitud

- Expande los caminos con menor número de estados

- $CP = CP + E$

- Garantiza el menor número de transiciones

Profundidad

- Expande los caminos con mayor número de estados

- $CP = E + CP$

- Suele ser más adecuada si la solución está alejada

• Búsqueda del camino óptimo: Branch & bound

Branch & bound

- Garantiza camino óptimo
- $CP = P(O(CP + E, c))$

Se va expandiendo por el coste más bajo hasta que su peso acumulado sea mayor que otro (posiblemente no explorado)

$O(C, c) \rightarrow$ ordenación de los caminos C según el coste c

$P(C) \rightarrow$ poda de la lista de caminos C .

Búsqueda informada

Debe ser ≥ 0 y $h(s) = 0$ si s es solución

• Búsqueda de una solución: Primero el mejor y método de la escalada. Cercanía S de ser solución

Primero el mejor

Amplitud

$$CP = O(CP + E, h)$$

Amplitud teniendo en cuenta heurística
Amplitud solo tiene en cuenta los pesos

Método de la escalada

Profundidad

$$CP = O(E, h) + CP$$

Solo se expande el de menor heurística (fórmula)
si hay empate dar o i.e.g. según el algoritmo

$O(P, h) \rightarrow$ Ordenación de los caminos P según el heurístico h .

• Búsqueda del camino óptimo: A^*

Cercanía S de ser solución óptima

Branch & bound

(peso acumulado + heurística del nodo)

A^*

$$CP = P(O(CP + E, c + h))$$

$O(C, c + h) \rightarrow$ Ordenación según heurística h y coste c .

$P(C) \rightarrow$ Poda de la lista de caminos C .

- Combina coste acumulado $c(C)$ y heurística h : $f(C) = c(C) + h(C)$
- Si h es admisible ($h(s) \leq$ coste real mínimo desde s a objetivo)

Algoritmo	Fórmula	Informado	Camino óptimo
Exploración en amplitud	$CP = CP + E$	No	—
Exploración en profundidad	$CP = E + CP$	No	—
Branch & bound	$CP = P(O(CP + E, c))$	No	Sí
Primero el mejor	$CP = O(CP + E, h)$	Sí	—
Método de la escalada	$CP = O(E, h) + CP$	Sí	—
A^*	$CP = P(O(CP + E, c + h))$	Sí	Sí

Computación Evolutiva

Rama de las metaheurísticas inspirada en la evolución biológica

Algoritmos genéticos:

- Ruleta ($\propto$ fitness)
- Torneo ($T \uparrow$ fitness)
- Selección por rango ($q_1 \dots q_n$)

- Cruce por un punto ($\vdots \times$)
- Cruce por orden ($\odot$)
- Cruce aritmético ($\vec{x}^* = \alpha \cdot \vec{x} + (1-\alpha)\vec{y}$)

- Mutación uniforme ($P_{mut} \cdot \text{gen}$)
- Mutación por intercambio (e_0)
- Mutación por inserción ($\square \rightarrow$)

Algoritmos genéticos (AG)

Metaheurística poblacional que mantiene una población de soluciones (cromosomas), aplica selección, cruce y mutación y reemplaza generaciones hasta un criterio de parada.

Son estocásticas, iterativas y trabajan bien cuando el espacio de búsqueda es grande o complejo.

Las metaheurísticas generalizan el concepto de heurística para buscar soluciones "buenas" a problemas de optimización.

Esquema General (GA)

1. Generar población inicial (aleatorio o semilla inteligente).
2. Decodificar cromosomas a soluciones (fenotipo).
3. Evaluar cada individuo con la función fitness ($q(i)$).
4. Seleccionar pares de padres (según probabilidad basada en fitness).
5. Cruzar/recombinar padres $\rightarrow$ hijos (operador de cruce).
6. Mutar genes de los hijos con probabilidad P_{mut} .
7. Selección ambiental: construir nueva población (generacional/estacionaria/elitismo).
8. Repetir hasta criterio de parada (número de generaciones, fitness objetivo, estancamiento del fitness).

Representación/Codificación de soluciones (AG)

- Codificación: cadenas de bits, enteros, reales o permutaciones según el problema. Elegir la codificación correcta es crítico: condiciona qué operadores son válidos y la facilidad de decodificar a una solución real.
- Decodificación: función que convierte cromosoma $\rightarrow$ solución correcta (fenotipo).

Función fitness

- Fitness mide calidad a maximizar (maximiza $q'(i) = \frac{1}{(1+\text{error})}$ para minimizar $q(i) = (1+\text{error})$)

- Para problemas con restricciones:

- Penalización fuerte: devolver 0 si se violan restricciones

- Penalización débil: restar o dividir por un coste de violaciones, usando K_j según importancia.

Operadores genéticos esenciales

Selección parental

- **Ruleta/selección estándar**: probabilidad proporcional a fitness (sensible a escalas extremas). $\rightarrow$ Normalización de probabilidades.

$$q'(i) = \frac{q(i) - \min_{j=0}^N q(j)}{\max_{j=0}^N q(j) - \min_{j=0}^N q(j)}$$

- Se elimina al peor individuo.
- Se aumenta la probabilidad de seleccionar los más adaptados.
- Riesgo: mínimos locales.

- **Torneo**: elegir τ individuos aleatoriamente y quedarse con el mejor (el de mayor fitness).

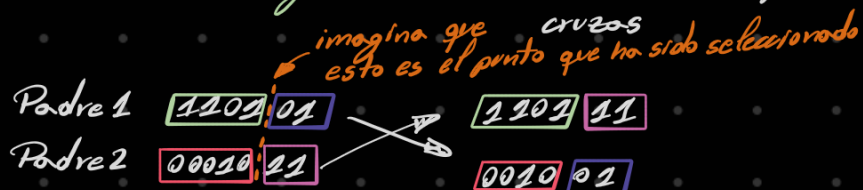
- **Selección por rango** (con probabilidad p).

- Se ordenan los individuos según $q(i)$.
- Se selecciona con probabilidad p desde el primer elemento hasta el último.

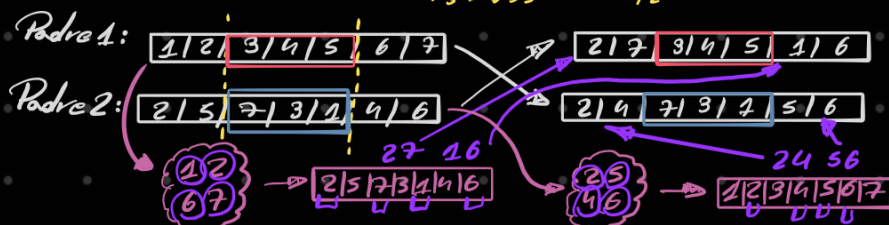
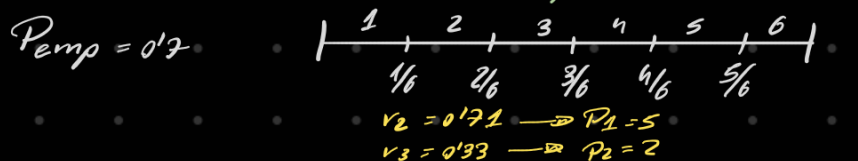
Cruce (crossover)/Emparejamiento

- **Cruce de un punto o N puntos**.

- Selecciona el punto de corte del cromosoma mediante elección equiprobable entre 1 y $L-1$. $\rightarrow$ y una vez con el punto los



- **Cruce por orden**: se eligen dos puntos de corte y se copian los valores entre dichos puntos de un padre en un hijo, rellenando los huecos con los valores del otro padre en el orden en que aparecen. $L=7$

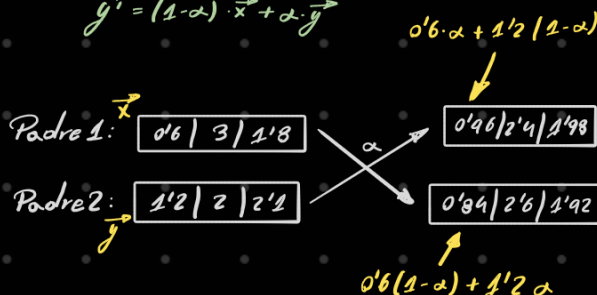


- **Cruce aritmético**: $\vec{x}$ e $\vec{y}$ como padres, se elige un valor aleatorio $\alpha \in [0, 1]$ y se calculan los hijos $\vec{x}' = \alpha \cdot \vec{x} + (1-\alpha) \cdot \vec{y}$

$$\vec{y}' = (1-\alpha) \cdot \vec{x} + \alpha \cdot \vec{y}$$

$P_{\text{emp}} = 0.7$

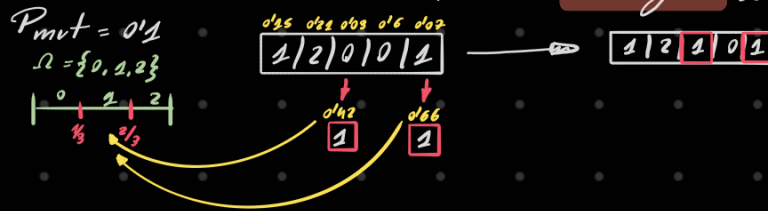
$\alpha = 0.14$



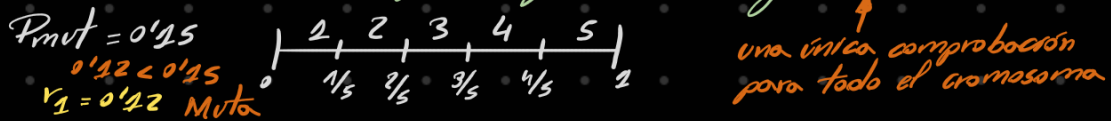
• Mutación

- Uniforme (reemplaza gen con valor aleatorio del alfabeto Ω).

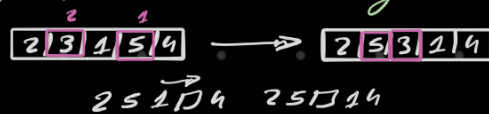
↑ se recorre toda la cadena mutando cada gen con probabilidad P_{mut}



- Mutación por intercambio: se eligen dos genes aleatorios y se intercambian los valores



- Mutación por inserción: se eligen dos genes y se inserta el valor del primero en la posición del segundo, desplazando el resto de genes.



• Reemplazo/selección ambiental

- Generacional: sustituye toda la población por hijos.

- Estacionario (steady-state): sustituye solo algunos individuos

- Elitismo: copia M mejores directamente para evitar pérdida de lo mejor.

